

Q.3

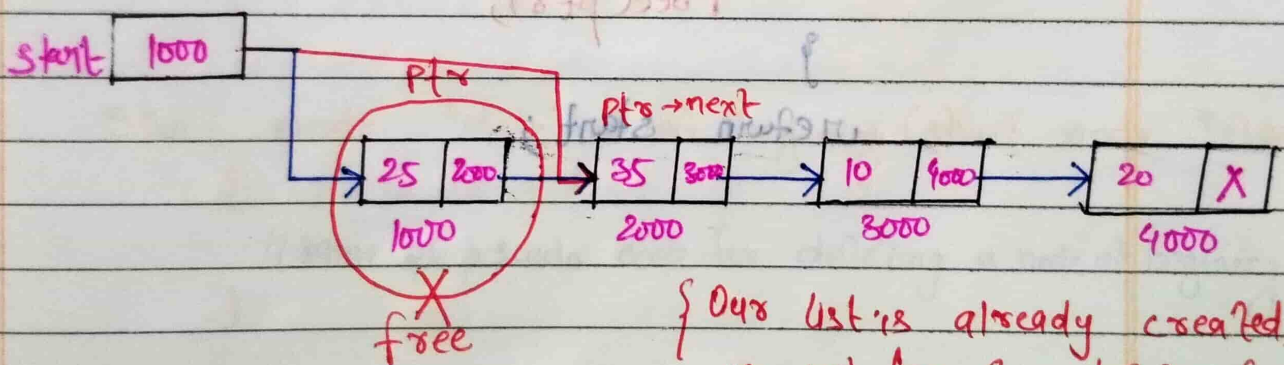
→ Deletion from a Linked List :-

Deletion in a linked list means removing a node from the list while maintaining the correct structure.

There are three main cases :-

1.) Deletion at the beginning :

Remove the first node and update the head pointer (start) to the next node.



Our list is already created, we just have to delete from at the beginning.

Algorithm :-

Step 1.) IF $start == NULL$
point "empty list"
goto step 5

2.) set $ptx = start$

3.) set $start = ptx \rightarrow next$

4.) $free(ptx)$

5.) Exit

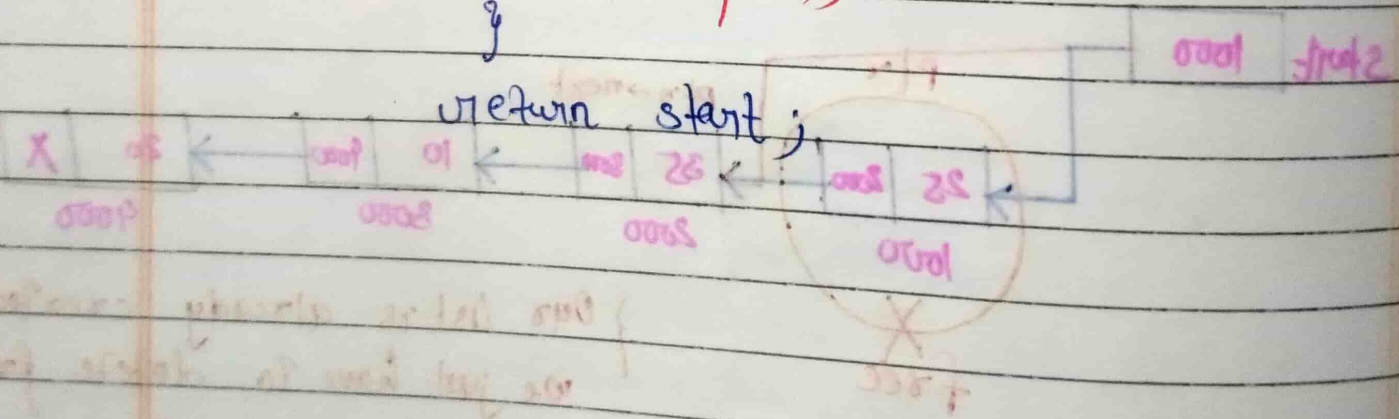
Pseudo code for deleting a node at beginning:

```

struct node *ptr;
if (start == NULL)
{
    printf("empty list");
}
else
{
    ptr = start;
    start = ptr->next;
    free(ptr);
}

```

return start;



// C program: (for) deleting a node at beginning

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *create_list(struct node *start)
```

```
{
```

```
    // same as create_list
```

```
}
```

```
struct node *deletion_at_beginning(struct node *start)
```

```
{
```

```
    // same as pseudo code for deleting a node at beginning
```

```
}
```

```
struct node *display_list(struct node *start)
```

```
{
```

```
    // same as display_list
```

```
}
```

```
int main()
```

```
{
```

```
    struct node *start = NULL;
```

```
    start = create_list(start);
```

```
    start = deletion_at_beginning(start);
```

```
start = display - list (start);
return 0;
```

```
}
```

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
{
```

```
int main()
```

```
{
    string s;
```

```
{
```

```
    cout << "Enter a string: ";
```

```
{
```

```
    s = "Akash";
```

```
{
```

```
    cout << "The string is: " << s << endl;
```

```
{
```

```
    return 0;
```

```
{
```

```
    cout << "The string is: " << s << endl;
```

```
{
```

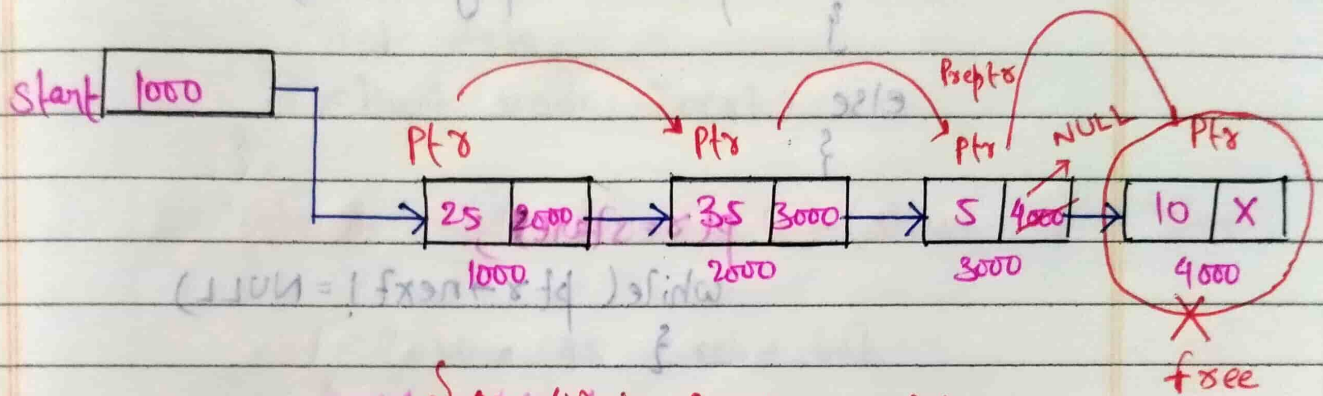
```
    return 0;
```

```
}
```


Q.4

2. Deletion at the end:

Traverse to the second-last node, set its next pointer to NULL, and delete the last node.



As list is already created, we just have to delete a node at the end.

Algorithm :-

Step 1. I.F (start == NULL)

print "empty list"

goto step 6

2. Set ptr = start

3. Repeat until ptr->next != NULL

3.1. preptr = ptr

3.2. ptr = ptr->next

4. preptr->next = NULL

5. free(ptr)

6. Exit

Pseudo code for deleting a node at end:

```
struct node *preptr, *ptr;
```

```
if (start == NULL)
```

```
{
```

```
printf("empty list");
```

```
}
```

```
else
```

```
{
```

```
ptr = start;
```

```
while (ptr->next != NULL)
```

```
{
```

```
preptr = ptr;
```

```
ptr = ptr->next;
```

```
}
```

```
preptr->next = NULL;
```

```
free(ptr);
```

```
}
```

```
return start;
```


// C program for deleting a node at end

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *create-list(struct node *start)
```

```
{
```

```
    // same as create-list
```

```
}
```

```
struct
```

```
struct node *deletion-at-end(struct node *start)
```

```
{
```

```
    // same as pseudo code for deletion a node
```

```
    // at end
```

```
}
```

```
struct node *display-list(struct node *start)
```

```
{
```

```
    // same as display-list
```

```
}
```

```
int main()
```

```
{
```

```
    struct node *start = NULL;
```

```

    start = create_list(start);
    start = deletion_at_end(start);
    start = display_list(start);
    return 0;
}

```

```

    if (start == NULL)
        return 0;
    struct tnode *temp = start;
    while (temp != NULL)
    {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

```

```

// Deletion at end of linked list
// If the list is empty, return 0
// If the list is not empty,
// traverse to the last node
// and set its next to NULL
// return the head of the list

```

```

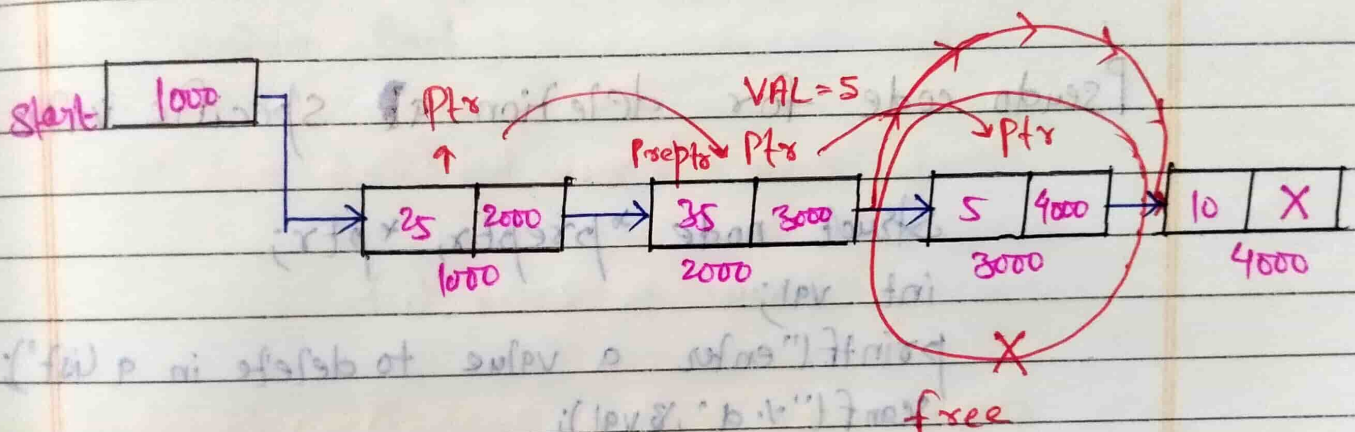
// Display the linked list
// If the list is empty, return 0
// If the list is not empty,
// traverse to the last node
// and print the data of each node
// return the head of the list

```


3.7 Deletion @ a specific value :

~~Find the node before the value, update its next pointer to skip the node, and delete the unwanted node.~~

Find the node and delete the node.



Algorithm :

- Step 1. > Enter a value to delete in a list
2. > IF (start == NULL)
"empty list"
goto step 8
3. > else,
set ptr = start
4. Repeat until ptr == NULL and ptr->data == VAL
4.1. > pnext = ptr
4.2. > ptr = ptr->next
5. IF (ptr == NULL)
print "value not found on list".

6) Else,
set $\text{preptr} \rightarrow \text{next} = \text{ptr} \rightarrow \text{next}$

7.) $\text{free}(\text{ptr})$

8. Exit

Pseudo code for deletion of specific value

```
struct node *preptr, *ptr;
```

```
int val;
```

```
printf("enter a value to delete in a list");
```

```
scanf("%d", &val);
```

```
if (start == NULL)
```

```
{
```

```
printf("empty list");
```

```
else
```

```
{
```

```
ptr = start;
```

```
while (ptr != NULL && ptr->data != val)
```

```
{
```

```
preptr = ptr;
```

```
ptr = ptr->next;
```

```
if (ptr == NULL)
```

```
{
```


printf("Value not found in list\n");

else
{

ptr->next = ptr->next;
free(ptr);

}

return start;

(ptr->next) if (ptr->next == NULL) {

printf("List is empty\n");

return ptr->next;

if (ptr->next == NULL) {

(ptr->next) if (ptr->next == NULL) {

printf("List is empty\n");

return ptr->next;

// C program for deletion a specific value

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *create_list(struct node *start)
```

```
{
```

```
    // same as create_list
```

```
}
```

```
struct node *deletion_a_specific_value(struct node
```

```
*start)
```

```
{
```

```
    // same as pseudo code for deletion
```

```
    // a specific value
```

```
}
```

```
struct node *display_list(struct node *start)
```

```
{
```

```
    // same as display_list
```

```
}
```

```
int main()
```

```
{
```



```

struct tnode *start = NULL;
start = create_list(start);
start = deletion_a_specific_value(start);
start = display_list(start);
return 0;
}

```

for 2 = 5 10 15 20

1000 = 1000 1000 1000 1000
++ for loop 1.5

for loop 1.5 1000 1000 1000

for loop 2.5 1000 1000 1000

→ Algorithm to count the total element in singly linked list:

Step 1: IF $start == NULL$
print "empty list"

Step 2: else,
set $ptr = start$

Step 3: Repeat until $ptr == NULL$
3.1. count++

Step 4: print count

Step 5: exit

Wag